Child processes

10 min read · Updated 24 Jul 2026 · Topic 63 of 70

Bookmark · Mark done

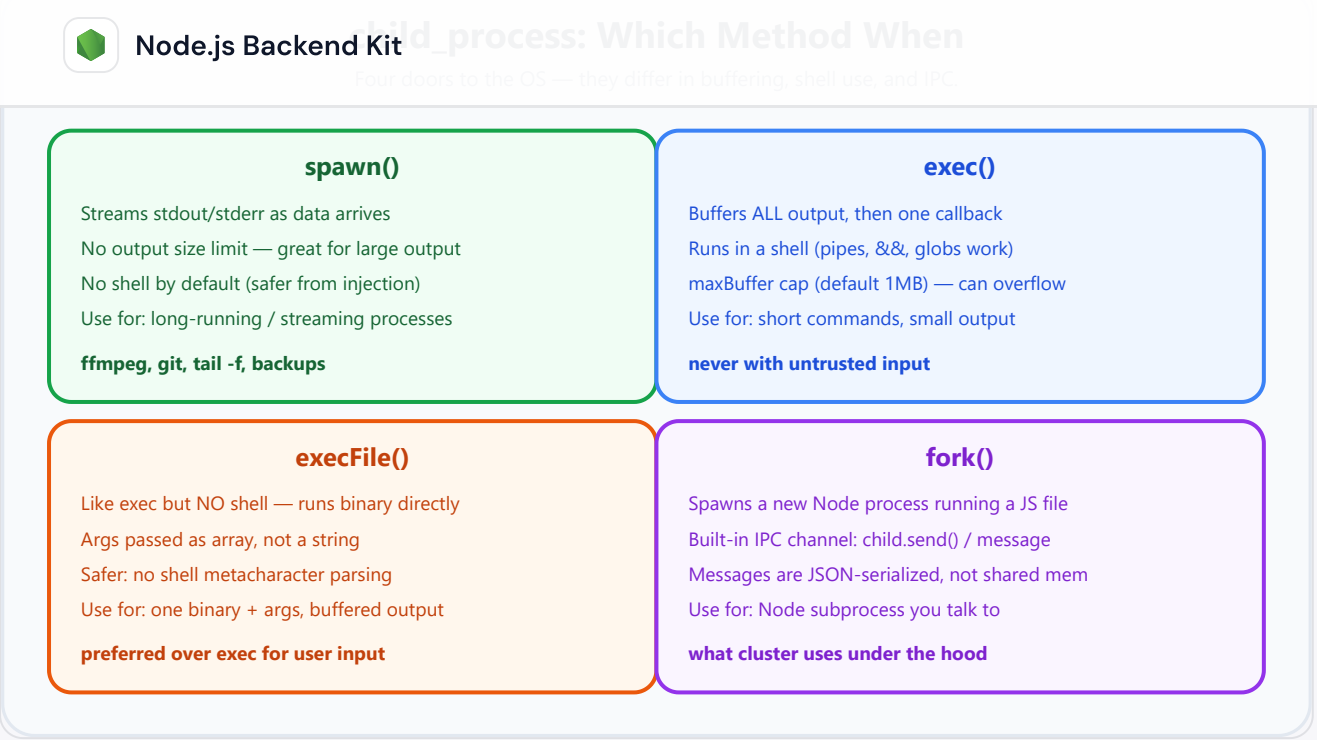
Why it matters

Not everything your service needs to do is JavaScript. You want to transcode a video with `ffmpeg`, run a Git command, shell out to a Python script, invoke ImageMagick, run a database backup, or call any of the thousands of mature command-line tools that already solve a problem better than you could re-implement it. The **child_process** module is Node's bridge to the operating system: it lets your app **launch and control external programs**, stream their output, feed them input, and get their exit status.

It is also how Node scales itself — `cluster` is built on `child_process.fork()` — and it is where one of the most dangerous security bugs in the ecosystem lives: **command injection**. Choosing `exec` vs `spawn` vs `execFile`, and whether a shell is involved, is not a stylistic preference; it decides whether an attacker who controls a filename can run arbitrary commands on your server. In interviews, being able to say *which* of the four methods to use and *why* — especially the shell/injection distinction — signals real production experience.

The concept, simply

child_process gives you four ways to start another program, and they differ along three axes: **does a shell get involved**, **is the output streamed or buffered**, and **is there a built-in messaging channel**.



spawn streams output with no shell; exec buffers all output through a shell; execFile runs a binary directly without a shell; fork starts a Node child with a built-in IPC channel.

- **spawn(cmd, args)** — launches a program and gives you its `stdout` / `stderr` as *streams*. Output arrives incrementally, so there's no size limit. No shell by default. Best for long-running or high-output processes (ffmpeg, `tail -f`, backups).
- **exec(cmdString)** — runs a command *through a shell* and *buffers* all output, handing it to a callback at the end. Convenient (pipes, `&&`, globs work), but capped by `maxBuffer` (1 MB default) and shell-based → injection risk. Best for short commands with small output.
- **execFile(file, args)** — like exec but *no shell*: it runs a binary directly with an args array. Safer against injection because shell metacharacters aren't interpreted. Buffered output like exec.
- **fork(modulePath)** — a specialization of spawn for running *another Node.js script*, with an automatic **IPC channel** so parent and child exchange messages via `child.send()` / `process.on('message')`. This is what cluster uses.

The mental shortcut: **spawn/execFile take an args array and skip the shell (safe); exec takes one command string and uses a shell (convenient but risky); fork is for Node-to-Node with messaging.** All of them create a genuine separate OS process with its own memory — so this is another way (besides cluster and worker_threads) to move heavy or blocking work off your event loop.

How it works

Every child process exposes three standard streams — `stdin`, `stdout`, `stderr` — plus lifecycle events: `'spawn'` (started), `'exit'` (process ended, with code/signal), `'error'` (failed to start, e.g. binary not found), and `'close'` (streams flushed). With `spawn` you subscribe to `child.stdout.on('data', ...)` to consume output as it flows; with `exec` / `execFile` Node accumulates it for you and calls back with `(err, stdout, stderr)`.

The shell distinction is the crux. `exec` effectively runs `/bin/sh -c "your string"`. The shell parses that string, so metacharacters like `;`, `|`, `&&`, `$()`, and backticks are *interpreted*. If any part of that string comes from user input, an attacker can inject commands: a "filename" of `a.txt; rm -rf /` becomes two commands. `spawn` / `execFile` with an args array pass arguments *directly* to the program via the OS, with no shell parsing, so a malicious "filename" is just a (harmless) literal argument. This is why the rule is: **never build a shell command string from untrusted input; use spawn/execFile with an args array.** (If you truly need shell features, `spawn(cmd, args, { shell: true })` exists — but then you own the injection risk again.)

maxBuffer is the other classic footgun. Because `exec` / `execFile` buffer all output in memory, they cap it (default 1 MB). A command that prints more than that is killed with an `ERR_CHILD_PROCESS_STDOUT_MAXBUFFER_EXCEEDED` error and truncated output — a silent, confusing failure in production. For anything that can produce large output, use `spawn` and stream.

IPC via fork. When you `fork` a Node module, Node opens an extra pipe as a message channel. `child.send(obj)` serializes `obj` (JSON/structured-clone) and delivers it to the child's `process.on('message')`, and vice versa. Like cluster, this is copy-based messaging, not shared memory. You can also pass a server handle or socket over this channel, which is precisely how cluster shares its listening socket with workers.

All variants also have **synchronous siblings** — `spawnSync`, `execSync`, `execFileSync`. These *block the event loop* until the child exits, so they're fine in build scripts and CLIs but should essentially never appear in a request handler.

Code examples

Stream a long-running external tool with `spawn` — output arrives incrementally, no buffer cap, no shell.

JAVASCRIPT Copy

```
import { spawn } from 'node:child_process';

// Note: command and args are SEPARATE — no shell parses this string.
const ffmpeg = spawn('ffmpeg', ['-i', 'input.mov', '-vcodec', 'h264', 'out

ffmpeg.stdout.on('data', (chunk) => process.stdout.write(chunk));
ffmpeg.stderr.on('data', (chunk) => process.stderr.write(chunk));   // ffm

ffmpeg.on('error', (err) => console.error('failed to start:', err.message)
ffmpeg.on('close', (code) => {
  console.log(code === 0 ? 'transcode done' : `ffmpeg exited ${code}`);
});
```

The safe way to pass user input: `execFile` (or spawn) with an args array — shell metacharacters can't break out.

JAVASCRIPT Copy

```
import { execFile } from 'node:child_process';
import { promisify } from 'node:util';
const run = promisify(execFile);

async function countLines(userPath) {
  // Even if userPath is "a.txt; rm -rf /", it's passed as ONE literal arg
  const { stdout } = await run('wc', ['-l', userPath], { timeout: 5000 });
  return Number(stdout.trim().split(/\s+/)[0]);
}

// DANGER — never do this with untrusted input:
//   exec(`wc -l ${userPath}`)   // shell parses ';' → command injection
```

Two-way messaging with `fork` — offload a job to a Node child and get the result over IPC.

```
// parent.js
import { fork } from 'node:child_process';

const child = fork('./report-worker.js');
child.send({ type: 'build-report', month: '2026-07' });
child.on('message', (msg) => {
  if (msg.type === 'done') { console.log('report ready:', msg.path); child
});

// report-worker.js
process.on('message', async (msg) => {
  if (msg.type === 'build-report') {
    const path = await buildHeavyReport(msg.month);   // CPU/IO heavy, off
    process.send({ type: 'done', path });
  }
});
```

Common mistakes & gotchas

- **Command injection via exec + user input.** The single most dangerous mistake. Interpolating untrusted data into an `exec` string lets attackers run arbitrary shell commands. Use `execFile` / `spawn` with an args array.
- **maxBuffer overflow with exec.** Output over 1 MB kills the child and truncates results. For large/streaming output use `spawn`.
- **Ignoring `stderr` and exit codes.** Many tools (ffmpeg, git) write progress/errors to stderr and signal failure via a non-zero exit code, not by throwing. Check `code` on `'exit'` / `'close'` and read stderr.
- **Confusing `'error'` with a non-zero exit.** `'error'` fires when the process couldn't *start* (binary missing, permission denied). A process that starts and then fails emits `'exit'` with a non-zero code — not `'error'`.
- **Using the `*Sync` variants in a server.** `execSync` / `spawnSync` block the entire event loop until the child finishes — fine in CLIs, catastrophic in request handlers.
- **Zombie/orphan processes.** If your Node process dies without cleaning up, long-running children can be orphaned. Track children and kill them on shutdown; consider

Large data over `send()` is serialized every time.

- **No timeout.** An external tool that hangs will hang your request forever. Always pass a `timeout` (and handle the kill).

Best practices

- **Default to `spawn` / `execFile` with an args array.** Only use a shell (`exec` or `shell: true`) when you truly need shell features *and* the whole command is trusted/static.
- **Never interpolate untrusted input into a command string.** Validate/allowlist arguments; pass them as array elements, not concatenated text.
- **Stream large output with spawn** instead of buffering with exec; set `maxBuffer` deliberately if you must buffer.
- **Always set a `timeout`** and handle the resulting kill signal; check exit codes and stderr for failure.
- **Track and clean up children** on `SIGTERM`/shutdown so you don't leak processes.
- **Prefer a native/library solution when one exists** — shelling out adds a process, a dependency on an installed binary, and an attack surface. Use child_process when a mature external tool genuinely beats reimplementing.
- **Use `fork` only for Node-to-Node** with messaging; for arbitrary binaries use spawn/execFile.

⭐ Interview lens

Q: spawn vs exec — what's the difference and when do you pick each?
A: `spawn` streams output (no size limit) and by default doesn't use a shell — best for long-running or high-output processes. `exec` runs through a shell and buffers all output into a callback (capped by `maxBuffer`, 1 MB) — convenient for short commands but risky with user input. Rule: spawn for streaming/large output, exec only for small trusted commands.

A: Don't build a shell command string from user input. Use `execFile` or `spawn` with the command and an *args array*, so no shell interprets metacharacters — a malicious "filename" like `a; rm -rf /` is passed as a single literal argument. Avoid `exec` and `shell: true` with untrusted data. (This is the trap they're fishing for.)

Q: What does fork give you that spawn doesn't?

A: fork is spawn specialized for launching another Node.js module, and it automatically sets up an IPC channel so parent and child exchange messages via `send()` / `'message'`. It's what cluster uses to fork and coordinate workers. spawn can run any binary but has no built-in messaging.

Q: You run `exec('git log')` on a huge repo and it fails. Why?

A: `exec` buffers all stdout in memory with a default `maxBuffer` of 1 MB. Large output exceeds it, so the child is killed and output truncated with a maxBuffer error. Fix: use `spawn` and stream stdout, or raise `maxBuffer` intentionally.

Q: Difference between the `'error'` event and a non-zero exit code?

A: `'error'` means the process failed to *start* (binary not found, no permission). A non-zero `exit` / `close` code means it started but the program itself failed. You must handle both — many candidates only check one.

Q: child_process vs worker_threads for offloading work?

A: child_process spawns a whole separate OS process (heavier, full isolation, good for running external programs or separate Node scripts). worker_threads runs threads inside the current process (lighter, can share memory via SharedArrayBuffer, best for CPU-bound JavaScript). For invoking ffmpeg/git → child_process; for a heavy JS calculation → worker_threads.

- **spawn** = stream output, no shell, no size cap → long-running / large output.
- **exec** = shell + buffered output (`maxBuffer` 1 MB) → short, *trusted* commands only.
- **execFile** = like exec but no shell, args array → safe way to buffer.
- **fork** = spawn a Node module + auto IPC (`send` / `message`) → what cluster uses.
- **Injection rule:** untrusted input → spawn/execFile with an **args array**, never an exec string.
- Events: `error` = failed to start; non-zero `exit` = ran but failed. Handle both + read stderr.
- Always set a **timeout**; kill children on shutdown; avoid `*Sync` in servers.

QUICK CHECK — no marks, just for you.

1. How do you prevent command injection when running an external program with user-supplied input?

- ◯ Escape the shell string manually before passing it to exec
- ◯ Use execFile or spawn with the command and an args array, so no shell interprets metacharacters
- ◯ Always set shell: true so the shell sanitises the input
- ◯ Wrap the exec call in a try/catch to catch injected commands

Check answer

2. You run exec("git log") on a huge repo and it fails partway. Why?

- ◯ exec buffers all stdout in memory with a default maxBuffer of 1 MB; large output exceeds it, so the child is killed and output truncated
- ◯ exec cannot run git commands, only shell built-ins

- [ ] git log requires spawn because it needs an IPC channel

Check answer

3. What does the child process 'error' event mean, versus a non-zero exit code?

- [ ] Both mean the same thing: the program ran and failed
- [ ] 'error' fires when the process fails to start (binary missing, permission denied); a non-zero exit code means it started but the program itself failed
- [ ] 'error' means the program failed; a non-zero exit means it could not start
- [ ] 'error' only fires for spawn, never for exec

Check answer

‹ Previous
Worker threads

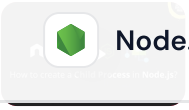
Next ›
Caching with Redis

LEARN MORE

ARTICLES & DOCS

- Node.js Child Process (spawn / exec / execFile / fork)
  nodejs.org
- Node.js Cluster module (built on child_process.fork)
  nodejs.org

VIDEOS



Hand-picked references for this lesson.